Lab 37 — Endpoint Detection and Response with osquery

> **SY0-701 Objective 4.5** — Given a scenario, modify enterprise capabilities to enhance security.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~20 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Turn a Linux host into an EDR-style sensor with osquery: query its live state with SQL, schedule a detection pack with `osqueryd`, plant a suspicious process and cron persistence yourself, hunt them down, then contain, eradicate and confirm the clean-up in the telemetry.

You'll build: An osquery installation from the official repository, a baseline of the host's attack surface, a scheduled `secplus-hunt` query pack writing JSON results to `/var/log/osquery/osqueryd.results.log`, and an evidence folder `/root/edr/` holding the hash of the captured malicious binary.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 14 (malware indicators) and Lab 34 (file integrity monitoring) are helpful background; tools installed in step 1.

Background

- **EDR** (endpoint detection and response) continuously records endpoint telemetry (processes, network sockets, persistence, file changes), detects suspicious **behaviour** rather than only known signatures, and supports **response** actions such as killing a process or isolating the host.
- **XDR** (extended detection and response) correlates EDR telemetry with network, email, identity and cloud signals in one platform, so a single incident is seen across every layer.
- **UBA/UEBA** (user and entity behaviour analytics) baselines what is normal for each user and device and flags deviations, such as a service account logging in interactively at 03:00. EDR supplies the endpoint data that UBA models learn from.
- **osquery** exposes the operating system as SQL tables (`processes`, `listening_ports`, `users`, `crontab`, `suid_bin`, `kernel_modules`). `osqueryi` is the interactive shell for hunting; `osqueryd` runs scheduled queries and logs **differential** results (rows `added` or `removed`), which a SIEM ingests.
- **Threat hunting** is proactive: start from a hypothesis (for example "malware runs from /tmp and persists via cron") and look for evidence before any alert fires.

Steps

1. Install osquery from the official repository

osquery is not in the Ubuntu archive, so add the project's signed apt repository. `jq` reads the JSON output and `netcat-openbsd` provides the binary you will disguise as malware in step 5.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y curl jq cron
netcat-openbsd ca-certificates
mkdir -p /etc/apt/keyrings /root/edr
```

```
curl -fsSL https://pkg.osquery.io/deb/pubkey.gpg -o /etc/apt/keyrings/osquery.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/osquery.asc]
https://pkg.osquery.io/deb deb main" > /etc/apt/sources.list.d/osquery.list
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y osquery
osqueryi --version
```

```
osqueryi version 5.x.x
```

2. Ask the endpoint questions with osqueryi

Every fact about the host is a table. Look at the system, the running processes, what is listening on the network and which accounts can log in.

```
osqueryi --json "SELECT hostname, cpu_brand, physical_memory FROM system_info;"
osqueryi --json "SELECT pid, name, path, resident_size FROM processes ORDER BY
resident_size DESC LIMIT 5;"
osqueryi --json "SELECT p.name, l.port, l.address, l.pid FROM listening_ports l JOIN
processes p USING (pid) WHERE l.port != 0;"
osqueryi --json "SELECT username, uid, shell FROM users WHERE shell NOT LIKE '%nologin'
AND shell NOT LIKE '%false';"
```

The `JOIN` maps each listening socket to the process that owns it, which is the first question an analyst asks about an unexpected port.

3. Baseline the persistence and privilege surface

Attackers persist with scheduled tasks, escalate with SUID binaries and hide with kernel modules. Record what is normal now so you can spot what changes later.

```
osqueryi --json "SELECT path, command FROM crontab;" | tee /root/edr/baseline-crontab.json
| jq length
osqueryi --json "SELECT path, username, permissions FROM suid_bin;" | tee
/root/edr/baseline-suid.json | jq length
osqueryi --json "SELECT name, size, status FROM kernel_modules ORDER BY name;" | tee
/root/edr/baseline-modules.json | jq length
osqueryi --json "SELECT path, username, permissions FROM suid_bin;" | jq -r '.[].path' |
head -8
```

The three numbers are your baseline counts of cron entries, SUID/SGID binaries and loaded kernel modules.

4. Schedule a detection pack with osqueryd

A query pack is a set of scheduled queries. `osqueryd` runs each one on its interval and writes only the rows that were added or removed since the last run.

```
mkdir -p /etc/osquery/packs /var/log/osquery
cat > /etc/osquery/packs/secplus-hunt.conf <<'EOF'
{
  "queries": {
    "suspicious_processes": {
      "query": "SELECT pid, name, path, cmdline, on_disk FROM processes WHERE on_disk = 0
OR path LIKE '/tmp/%' OR path LIKE '/dev/shm/%';",
```

```
      "interval": 30,
      "description": "Processes running from temp paths or whose binary was deleted"
    },
    "listening_ports": {
      "query": "SELECT p.name, p.path, l.port, l.address FROM listening_ports l JOIN
processes p USING (pid) WHERE l.port != 0;",
      "interval": 30,
      "description": "New network listeners"
    },
    "crontab": {
      "query": "SELECT path, command FROM crontab;",
      "interval": 30,
      "description": "Cron persistence"
    },
    "suid_binaries": {
      "query": "SELECT path, username, permissions FROM suid_bin;",
      "interval": 60,
      "description": "New SUID or SGID binaries"
    },
    "kernel_modules": {
      "query": "SELECT name, size FROM kernel_modules;",
      "interval": 60,
      "description": "Kernel module changes"
    }
  }
}
EOF
cat > /etc/osquery/osquery.conf <<'EOF'
{
  "options": {
    "config_plugin": "filesystem",
    "logger_plugin": "filesystem",
    "logger_path": "/var/log/osquery",
    "schedule_splay_percent": 10,
    "utc": "true"
  },
  "packs": {
    "secplus-hunt": "/etc/osquery/packs/secplus-hunt.conf"
  }
}
EOF
jq empty /etc/osquery/osquery.conf /etc/osquery/packs/secplus-hunt.conf && echo "JSON
valid"
osqueryd --config_check --config_path=/etc/osquery/osquery.conf && echo "osquery config
OK"
systemctl enable osqueryd >/dev/null 2>&1; systemctl restart osqueryd
sleep 40; systemctl is-active osqueryd
jq -r '.name' /var/log/osquery/osqueryd.results.log | sort | uniq -c
```

```
active
      5 pack_secplus-hunt_crontab
     .. pack_secplus-hunt_listening_ports
```

The first run of each query logs every current row as `added`; after that only changes appear.

5. Plant a suspicious process and cron persistence

Simulate a common Linux intrusion on your own playground: a renamed copy of netcat hidden in `/tmp` as a fake kernel worker, listening on port 4444, with its binary deleted to frustrate investigators, a cron entry to restart it, and a SUID shell left behind for privilege escalation.

```
cp "$(readlink -f "$(command -v nc)")" /tmp/.kworker-update
nohup /tmp/.kworker-update -lk 4444 </dev/null >/dev/null 2>&1 &
sleep 1; rm -f /tmp/.kworker-update
echo '*/5 * * * * root /tmp/.kworker-update -lk 4444 >/dev/null 2>&1' >
/etc/cron.d/kworker-update
cp /bin/bash /usr/local/bin/.dbus-helper && chmod 4755 /usr/local/bin/.dbus-helper
ss -tlnp | grep 4444
```

None of these are dangerous outside the lab: the listener accepts connections on your own host only, and the cron command points at a file that no longer exists.

6. Hunt for the indicators

Test the hypothesis "something is running from a temp path, listening, and persisting" with targeted queries, then compare the SUID list against the baseline from step 3.

```
osqueryi --json "SELECT pid, name, path, cmdline, on_disk FROM processes WHERE on_disk = 0
OR path LIKE '/tmp/%' OR path LIKE '/dev/shm/%';"
osqueryi --json "SELECT p.pid, p.path, l.port, l.address FROM listening_ports l JOIN
processes p USING (pid) WHERE p.on_disk = 0 OR p.path LIKE '/tmp/%';"
osqueryi --json "SELECT path, command FROM crontab WHERE command LIKE '%/tmp/%' OR command
LIKE '%/dev/shm/%';"
osqueryi --json "SELECT path, username, permissions FROM suid_bin;" | jq -r '.[].path' |
sort > /root/edr/now-suid.txt
jq -r '.[].path' /root/edr/baseline-suid.json | sort | comm -13 - /root/edr/now-suid.txt
```

```
"path":"/tmp/.kworker-update", "on_disk":"0", "cmdline":"/tmp/.kworker-update -lk 4444"
"path":"/etc/cron.d/kworker-update", "command":"root /tmp/.kworker-update -lk 4444 ..."
/usr/local/bin/.dbus-helper
```

`on_disk = 0` (a running binary that no longer exists on disk), an executable in `/tmp`, an unusual listening port and fresh cron persistence together are a strong malware signal.

7. Read the detections in the osqueryd telemetry

The scheduled pack caught the same changes on its own. This JSON stream is what an EDR agent ships to the SIEM (Lab 33) for alerting and correlation.

```
sleep 40
jq -c 'select(.action=="added" and (.name|test("suspicious_processes|suid_binaries"))) |
{time:.calendarTime, query:.name, row:.columns}' /var/log/osquery/osqueryd.results.log |
tail -4
jq -c 'select(.action=="added") | select(.columns|tostring|test("kworker|4444")) |
{query:.name, row:.columns}' /var/log/osquery/osqueryd.results.log | tail -4
```

Each line carries the query name, a timestamp, `action:"added"` and the offending row, ready for a SIEM rule such as "alert on any added row from suspicious_processes".

8. Respond: preserve evidence, contain, eradicate and confirm

Act as the EDR response console would: capture the deleted binary from `/proc` before killing the process (evidence first), hash it, kill it, remove the persistence and the SUID shell, then confirm the hunt queries come back empty and the telemetry records the removals.

```
PID=$(osqueryi --json "SELECT pid FROM processes WHERE path LIKE '/tmp/.kworker-update%';"
| jq -r '.[0].pid')
echo "Malicious PID: $PID"
cp "/proc/$PID/exe" /root/edr/kworker-update.evidence && chmod 400 /root/edr/kworker-
update.evidence
sha256sum /root/edr/kworker-update.evidence | tee /root/edr/evidence.sha256
kill "$PID"
mv /etc/cron.d/kworker-update /root/edr/cron-kworker-update.evidence
mv /usr/local/bin/.dbus-helper /root/edr/dbus-helper.evidence && chmod 000 /root/edr/dbus-
helper.evidence
osqueryi --json "SELECT pid, path FROM processes WHERE on_disk = 0 OR path LIKE '/tmp/%';"
sleep 70
jq -r 'select(.action=="removed") | .name' /var/log/osquery/osqueryd.results.log | sort |
uniq -c
```

The evidence hash matches the original netcat binary, which tells the investigator exactly what the attacker dropped. In a commercial EDR the same playbook (isolate host, kill process, quarantine file) is one click, and an XDR platform would also correlate it with the firewall and identity alerts for the same host and user.

Test it

```
osqueryi --json "SELECT pid, path FROM processes WHERE on_disk = 0 OR path LIKE '/tmp/%';"
osqueryi --json "SELECT path, command FROM crontab WHERE command LIKE '%/tmp/%';"
jq -r 'select(.name=="pack_secplus-hunt_suspicious_processes") | .action'
/var/log/osquery/osqueryd.results.log | sort | uniq -c
cat /root/edr/evidence.sha256
```

Expected: the first two queries return `[]`, the results log shows at least one `added` and one `removed` event for `suspicious_processes`, and the evidence hash file contains a 64-hex-character SHA-256.

Troubleshooting

Symptom	Fix
`apt-get install osquery` reports `NO_PUBKEY` or "not signed"	The key download failed; re-run the `curl ... -o /etc/apt/keyrings/osquery.asc` line, check it starts with `-----BEGIN PGP PUBLIC KEY BLOCK-----`, then `apt-get update` again.
`/var/log/osquery/osqueryd.results.log` does not exist	The daemon has not run a query yet or failed to start; check `systemctl status osqueryd` and `journalctl -u osqueryd -n 20`, fix any JSON error, then wait 40 seconds.
No `removed` events after step 8	Differential results only appear on the next interval; wait another 60 seconds and re-run the last `jq` line.

Challenge

Add a scheduled query to the pack that detects a new account with UID 0 or a new user with a login shell, create a test user with `useradd -m -s /bin/bash backdoor`, and show the `added` row in the results log. Then explain which UBA signal would make that account suspicious even if the creation looked legitimate.

Reflection

Signature antivirus would not have flagged a renamed copy of a legitimate netcat binary. Which behaviours made it suspicious, and why does that make EDR better suited to living-off-the-land and fileless attacks?

Exam tips

- Antivirus matches known signatures; **EDR** records behaviour on the endpoint and can respond (kill, quarantine, isolate); **XDR** extends that correlation across endpoint, network, email, identity and cloud.
- **UBA/UEBA** detects anomalies against a learned baseline of each user or entity (impossible travel, unusual hours, data volume) — it catches insider threats and compromised credentials that rules miss.
- Indicators of compromise on Linux: processes running from `/tmp` or `/dev/shm`, deleted binaries still running, unexpected listeners, new cron entries and new SUID files.
- Preserve evidence before eradication: capturing the binary and its hash first supports the investigation and chain of custody (objective 4.8).

References

- osquery documentation and schema — https://osquery.readthedocs.io and https://osquery.io/schema
- osquery query packs (incident-response, ossec-rootkit) — https://github.com/osquery/osquery/tree/master/packs
- MITRE ATT&CK T1053.003 Scheduled Task/Job: Cron — https://attack.mitre.org/techniques/T1053/003/
- MITRE ATT&CK T1548.001 Setuid and Setgid — https://attack.mitre.org/techniques/T1548/001/
- NIST SP 800-61 Computer Security Incident Handling Guide — https://csrc.nist.gov/pubs/sp/800/61/r2/final

---